DS 593 Project Proposal

1 Problem Statement & Motivation

Problem: We wish to combine aspects of the DOG from K9DB into the RuleKeeper framework to create a more complete and comprehensive GDPR compliance model for well-intentioned developers.

Research Question: We will additionally investigate whether DOG can automatically derive some of RuleKeeper's ownership-related policies and compare this with manually specifying them in terms of correctness, effort, and overhead.

Motivation: This is important because it may aid in the creation of a more complete compliance framework for developers. It will also allow us to examine how overhead changes when larger and more comprehensive compliance frameworks are applied to online systems.

2 Idea

High-Level Approach: Our idea is to combine aspects of K9DB with RuleKeeper. We will use a database together with the RuleKeeper framework. Since RuleKeeper is built for MERN, we will use the MERN stack for our work.

Proposed Design: Ideally, we will build the DOG over MongoDB, incorporating existing technology from RuleKeeper where applicable. The DOG will represent and infer ownership relationships, while RuleKeeper will enforce the resulting policies.

Optimization: If we find that certain aspects of the two systems overlap, we will test different approaches to reduce redundancy and unnecessary performance overhead.

3 Related Work / Relevant Techniques

Related Systems: Our project builds on both RuleKeeper and K9DB. K9DB provides the DOG abstraction for representing data ownership, while RuleKeeper provides the policy enforcement framework.

Implementation: Since these systems are built in different languages, we will likely have to adapt some code in order to integrate their ideas. This will involve applying and developing knowledge of C++, at least enough to understand the K9DB code, as well as coding in JavaScript.

Relevant Skills: In adapting the DOG, we will develop and apply skills in database design and ensure that we have a good understanding of both the theory and implementation of the systems we are adapting.

4 Evaluation Plan

We will evaluate our system around three main questions:

- **Correctness**
  - **Question:** Do DOG-derived policies behave consistently with manual RuleKeeper policies?
  - **Measure:** Agreement rate, false allows, and false denies.
- **Performance Overhead**
  - **Question:** How much overhead does the DOG introduce?
  - **Measure:** Latency, throughput, and policy derivation time.
- **Developer Effort**
  - **Question:** Does the DOG reduce manual policy specification?
  - **Measure:** Number of manually specified ownership relationships and policies with versus without DOG.

5 Project Plan

5.1 Main Deliverables

- Working DOG + RuleKeeper prototype
- DOG construction and ownership inference
- DOG-to-RuleKeeper policy derivation
- Demo application and workload
- Evaluation results

5.2 Minimum Success

A working prototype that can derive and enforce basic transitive ownership policies through RuleKeeper and evaluate them against manually specified policies.

5.3 Evaluation Outputs

- **Correctness:** Whether DOG-derived policies and manual RuleKeeper policies produce consistent enforcement behavior.
- **Developer Effort:** How much manual policy specification can be reduced?
- **Overhead:** How much performance overhead does DOG construction, inference, and enforcement add?

5.4 Division of Work

- **Puchen:** DOG representation and ownership inference.
- **Sachiel:** RuleKeeper integration and benchmark/evaluation infrastructure.
- **Together:** System design, integration, experiments, and final report.

5.5 Reach Goals

- **Shared Ownership:** Support data owned by multiple users.
- **Multiple Ownership Paths:** Handle ownership derived through different paths in the DOG.
- **Policy Conflicts:** Resolve conflicts between DOG-derived and manually specified RuleKeeper policies.
- **Scalability:** Evaluate the system on larger and more complex DOGs.